Vincenzo Sguera 1170412

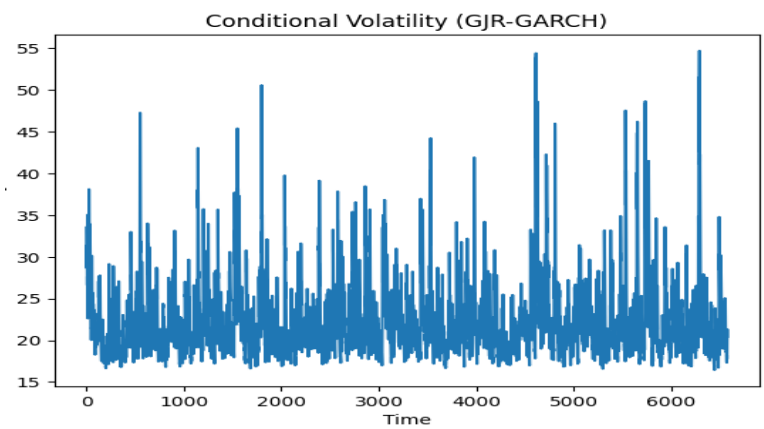
Exercise 1 – Time-Varying Volatility Analysis of a Financial Asset

The first worksheet contains the log-price series of a financial asset. Returns are computed as first differences of the log price. The graphical inspection of returns shows clear volatility clustering: large shocks tend to be followed by periods of persistent high volatility, while serene periods are followed by low volatility. This is typical of financial time series.

The Ljung–Box test on returns reports p-value = 0.235, suggesting no significant linear autocorrelation in the mean equation. Conversely, the Ljung–Box test on squared returns gives p-value ≈ 1.50×10^-131, strongly rejecting the null of no autocorrelation in the second moment. This provides strong evidence of conditional heteroskedasticity and motivates the use of ARCH/GARCH models.

```
      lb_stat  lb_pvalue
10  12.807669   0.234625
       lb_stat      lb_pvalue
10  642.316232  1.495866e-131
```

The Jarque–Bera p-value equal to 0 indicates strong non-normality and fat tails, another standard feature of asset returns. The estimated GARCH-type specification captures the persistence of volatility dynamics. Therefore, the suitable conclusion is that returns exhibit pronounced time-varying volatility and persistent volatility clustering (as can be seen in the "Conditional volatility" plot).



Also, we applied a GARCH(1,1), a t-GARCH and an asymmetric t-GARCH; the last one seems to be the best performer (the asymmetric t-GARCH is preferred because it captures both fat tails and leverage effects, as confirmed by better information criteria (AIC/BIC) and residual diagnostics), here the summary of statistics:

```
                     Constant Mean - GJR-GARCH Model Results
==============================================================================
Dep. Variable:                     LogPrice   R-squared:                   0.000
Mean Model:                   Constant Mean   Adj. R-squared:              0.000
Vol Model:                        GJR-GARCH   Log-Likelihood:           -29456.8
Distribution:      Standardized Student's t   AIC:                      58925.5
Method:                 Maximum Likelihood   BIC:                      58966.3
                                             No. Observations:            6580
Date:                     Fri, Apr 03 2026   Df Residuals:                6579
Time:                             17:29:34   Df Model:                       1
                                 Mean Model
=========================================================================
                 coef    std err          t      P>|t|  95.0% Conf. Int.
-------------------------------------------------------------------------
mu             0.3307      0.245      1.352      0.176 [ -0.149,  0.810]
                              Volatility Model
==============================================================================
                 coef    std err          t      P>|t|       95.0% Conf. Int.
------------------------------------------------------------------------------
omega         55.4898      9.457      5.868  4.416e-09      [ 36.955, 74.025]
alpha[1]       0.1277  1.689e-02      7.559  4.053e-14    [9.460e-02,  0.161]
gamma[1]      -0.0347  1.817e-02     -1.912  5.590e-02 [-7.035e-02,8.745e-04]
beta[1]        0.7817  2.708e-02     28.859 3.857e-183      [  0.729,  0.835]
                             Distribution
=========================================================================
                 coef    std err          t      P>|t|  95.0% Conf. Int.
-------------------------------------------------------------------------
nu             6.7012      0.535     12.529  5.177e-36 [  5.653,  7.750]
=========================================================================
```

Exercise 2 – Interconnected and Time-Varying Volatility Across Three Assets

The static correlation matrix of returns indicates positive contemporaneous dependence among the three assets: corr(1,2)=0.186, corr(1,3)=0.312, corr(2,3)=0.259. This result suggests that there is a moderate co-movement in returns.

The correlation matrix of standardized residuals remains positive (0.172, 0.269, 0.252), indicating residual co-movement even after filtering the conditional mean. More importantly, the correlation of squared standardized residuals shows remaining volatility linkages, especially between Asset 1 and Asset 2 (0.321), which is evidence of interconnected volatility paths.

```
Correlation matrix:
          Price1    Price2    Price3
Price1  1.000000  0.186442  0.312364
Price2  0.186442  1.000000  0.258580
Price3  0.312364  0.258580  1.000000

Correlation matrix of squared returns (volatility dependence):
          Price1    Price2    Price3
Price1  1.000000  0.312625  0.277384
Price2  0.312625  1.000000  0.049609
Price3  0.277384  0.049609  1.000000
```

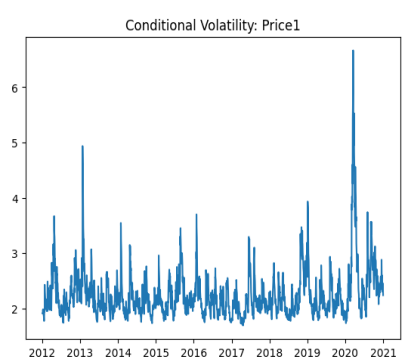
```
Correlation of squared standardized residuals
          Price1    Price2    Price3
Price1  1.000000  0.321488  0.105368
Price2  0.321488  1.000000  0.025849
Price3  0.105368  0.025849  1.000000
```

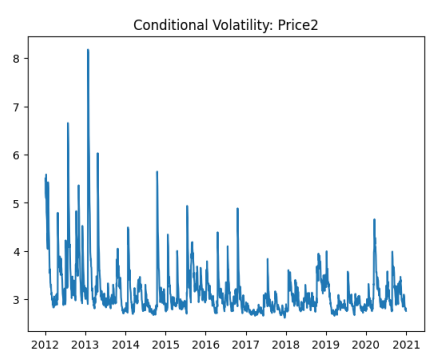
The Ljung–Box statistic for Price1 is highly significant (p-value ≈ 1.75×10^-18), confirming persistence in higher moments and strong volatility clustering. The graphical volatility paths suggest that shocks tend to propagate across assets over time, supporting the presence of volatility spillovers and synchronized risk regimes.

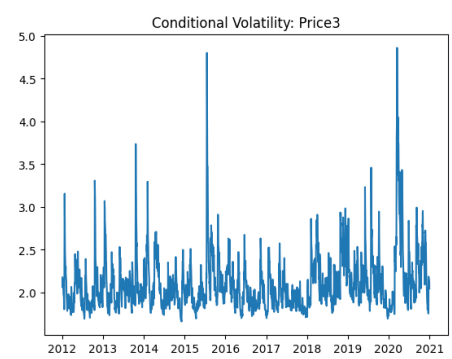
```
Ljung-Box test for Price1:
       lb_stat     lb_pvalue
10  107.438310  1.751075e-18
20  117.131141  9.666265e-16
```

The positive correlations in squared standardized residuals indicate that periods of high volatility tend to occur simultaneously across assets, providing evidence of volatility spillovers and common risk factors. Overall, the evidence supports the presence of interconnected and time-varying volatility dynamics. This suggests that shocks to one asset can propagate to others, leading to synchronized volatility regimes.

The conditional volatility plots show that volatility spikes tend to occur simultaneously across assets, indicating the presence of common shocks and volatility spillovers.







Significant cross-correlations in squared returns indicate that volatility shocks in one asset can precede volatility increases in another. Significant cross-correlations at non-zero lags indicate that volatility shocks propagate across assets over time, suggesting the presence of lead–lag spillover effects. See the plots on page 4.

Exercise 3 – VAR Model and Structural Impulse Response Analysis

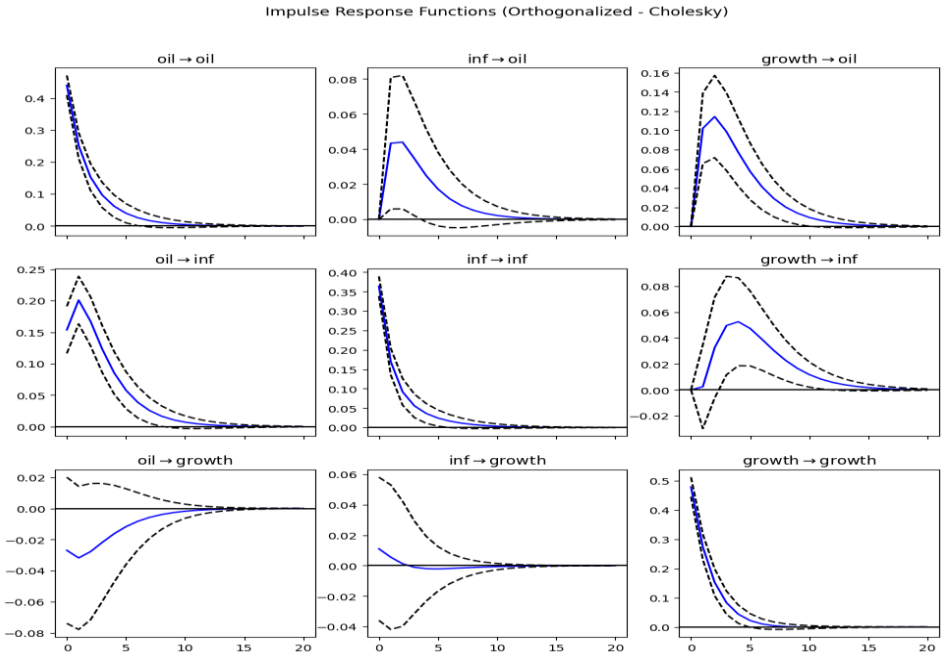
Augmented Dickey–Fuller tests strongly reject the unit-root null for all variables (oil prices, inflation, and GDP growth), indicating that the series are stationary and can be modeled using a VAR in levels. The optimal lag length is selected according to standard information criteria, and a VAR(p) model is estimated.

Structural impulse response functions are obtained using a Cholesky decomposition with ordering (oil, inflation, growth), assuming that oil prices are contemporaneously exogenous within the system.

The impulse response analysis shows that a positive oil price shock generates an immediate and significant increase in inflation, consistent with cost-push pressures in a small open economy. This effect gradually decays over time, indicating partial mean reversion.

GDP growth responds negatively to an oil price shock, with the effect materializing after a short delay. This reflects the increase in production costs and the contractionary impact on economic activity.

The impulse responses also indicate that oil shocks are persistent, while their effects on inflation and growth are transitory but economically meaningful.



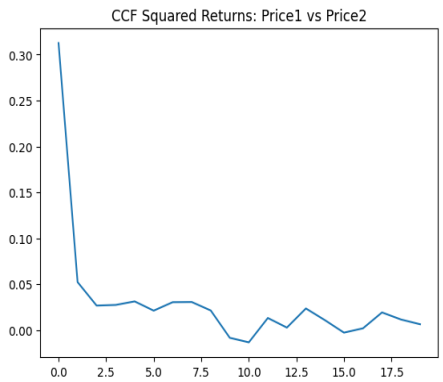
The forecast error variance decomposition (FEVD) shows that, although own shocks dominate in the short run, oil price shocks account for an increasing share of the variability in both inflation and GDP growth at longer horizons. See the plots on page 4.

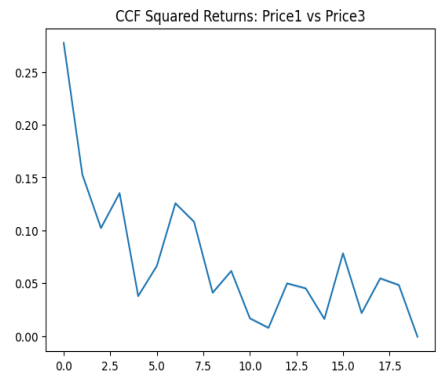
In addition, the forecast error variance decomposition (FEVD) provides further insights into the importance of oil shocks. While oil prices are mainly driven by their own innovations, oil shocks explain an increasing share of inflation variability over time, reaching approximately 40% at longer horizons. This highlights the strong influence of oil price dynamics on inflation. In contrast, GDP growth remains largely driven by its own shocks, with oil shocks playing a relatively minor role.

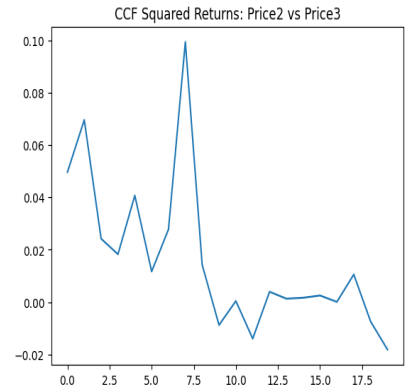
Overall, the results are consistent with a supply-side transmission mechanism, whereby oil price increases lead to higher inflation and subsequently lower economic growth.

Appendix – Additional Plots

EXERCISE 2







EXERCISE 3

```
Forecast Error Variance Decomposition:
FEVD for oil
          oil       inf    growth
0    1.000000  0.000000  0.000000
1    0.954342  0.006988  0.038670
2    0.911323  0.012399  0.076278
3    0.883520  0.015287  0.101193
4    0.867713  0.016654  0.115633
5    0.859290  0.017269  0.123440
6    0.854991  0.017538  0.127471
7    0.852867  0.017653  0.129480
8    0.851845  0.017702  0.130453
9    0.851364  0.017723  0.130913

FEVD for inf
          oil       inf    growth
0    0.151146  0.848854  0.000000
1    0.283762  0.716211  0.000027
2    0.349869  0.646079  0.004051
3    0.377633  0.609955  0.012412
4    0.387865  0.590861  0.021275
5    0.391051  0.580670  0.028279
6    0.391744  0.575282  0.032973
7    0.391706  0.572490  0.035804
8    0.391533  0.571077  0.037389
9    0.391390  0.570380  0.038230

FEVD for growth
          oil       inf    growth
0    0.003171  0.000530  0.996299
1    0.005707  0.000499  0.993795
2    0.007643  0.000467  0.991890
3    0.008896  0.000460  0.990643
4    0.009622  0.000469  0.989909
5    0.010011  0.000482  0.989507
6    0.010209  0.000492  0.989299
7    0.010305  0.000499  0.989196
8    0.010350  0.000504  0.989146
9    0.010371  0.000506  0.989123
```

Appendix – Python Code

Ex 1

```
#Exercise 1
#First, we import the required packages for the analysis
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

#Then we use the google.colab package to upload the file stored locally
(select the file to upload)
from google.colab import files
uploaded = files.upload()

#Load the excel file and the 1st sheet
df = pd.read_excel('Sguera_Vincenzo.xlsx', sheet_name='Ex.1')

#Assuming the first column is time and the second one is log price, then we
clean the dataset
df.columns = ['Date', 'LogPrice']

#Then we create the LogReturns and clean the dataset dropping the first row
returns = df["LogPrice"].diff().dropna()
df = df.drop(df.index[0])


#Checking if the dataset is correct
print(df)

#Plotting the price for a graphical inspection
plt.figure()
plt.plot(df['Date'], df['LogPrice'])
plt.title('Historical price')
plt.show()

#Then we plot only the returns
plt.figure()
plt.plot(df['Date'], returns)
plt.title('Returns')
plt.show()


#Now we plot the ACF of returns and squared returns
from statsmodels.graphics.tsaplots import plot_acf

plot_acf(returns, lags=40)
```

```
plt.title("ACF Returns")
plt.show()

plot_acf(returns**2, lags=40)
plt.title("ACF Squared Returns")
plt.show()


#Now we will analyze the autocorrelation in the series through the Ljung-Box
from statsmodels.stats.diagnostic import acorr_ljungbox

#On returns
lb_returns = acorr_ljungbox(returns, lags=[10], return_df=True)

#On squared returns
lb_sq = acorr_ljungbox(returns**2, lags=[10], return_df=True)

print(lb_returns)
print(lb_sq)


#We may also use a Jarque-Bera test in order to spot if there are fat tails
from scipy.stats import jarque_bera

jb_stat, jb_p = jarque_bera(returns)
print("JB p-value:", jb_p)


#We use the QQ plot to spot the distribution of observations
import scipy.stats as stats

#First we plot it against a normal distrribution
stats.probplot(returns**2, dist="norm", plot=plt)
plt.title("QQ Plot - Normal")
plt.show()

#then we plot it against a t distribution (5 degree of fredom)
stats.probplot(returns**2, dist="t", sparams=(5,), plot=plt)
plt.title("QQ Plot - t (df=5)")
plt.show()


#Now we'll start to model volatility using a GARCH(1, 1) model
from arch import arch_model

#Creating the model
model = arch_model(returns*100, vol='Garch', p=1, q=1, dist='normal')
res = model.fit()
```

```
#Printing the summary of the model
print(res.summary())


#Now we'll use a t-GARCH
model_t = arch_model(returns*100, vol='Garch', p=1, q=1, dist='t')
res_t = model_t.fit()

#Printing the summary of the model
print(res_t.summary())


#Now we build an asymmetric GARCH model in order to capture the so-called
Leverage effect
model_gjr = arch_model(returns*100, vol='GARCH', p=1, o=1, q=1, dist='t')
res_gjr = model_gjr.fit()

#Printing the summary of the model
print(res_gjr.summary())


#Plotting conditional volatility
plt.figure()
plt.plot(res_gjr.conditional_volatility)
plt.title("Conditional Volatility (GJR-GARCH)")
plt.xlabel("Time")
plt.ylabel("Volatility")
plt.show()


#We proceed in our analysis going through the analysis of the "Half-life" of
volatility for asymmetric GARCH
alpha = res.params.get('alpha[1]')
beta = res.params.get('beta[1]')
gamma = res.params.get('gamma[1]', 0)  # default = 0 if not found

persistence = alpha + beta + gamma / 2
half_life = np.log(0.5) / np.log(persistence)

#This is an approximation, it's difficult to find the true value
print("Half-life:", half_life)


#Then we want to analyse the residuals
residuals = res_gjr.resid
std_residuals = res_gjr.std_resid

#We will do a Ljung Box test
print(acorr_ljungbox(std_residuals, lags=[10, 20], return_df=True))
```

```
print(acorr_ljungbox(std_residuals**2, lags=[10, 20], return_df=True))

#Then we plot them
plt.figure()
plt.plot(res_gjr.std_resid)
plt.title("Standardized Residuals")
plt.xlabel("Time")
plt.ylabel("Std Residuals")
plt.show()
```

Ex 2

```
#Exercise 2
#Loading the Excel sheet Ex 2
df = pd.read_excel('Sguera_Vincenzo.xlsx', sheet_name='Ex.2')

#We convert the date
df['obs'] = pd.to_datetime(df['obs'], dayfirst=True)
df = df.set_index('obs')

#Compute log returns using np
returns = np.log(df / df.shift(1)).dropna()

#We print the head to verify the correct format
print(returns.head())


#Now we'll print the correlation matrix of returns (static interdependence)
print("\nCorrelation matrix:")
print(returns.corr())

#Correlation of squared returns (VOLATILITY COMOVEMENT)
print("\nCorrelation matrix of squared returns (volatility dependence):")
print((returns**2).corr())


#We will show the time variation between prices
#By using the enumerate function we are able to analyse the time varying
dependence (rolling correlations)
#We impose a window of 60 days
window = 60

pairs = [('Price1', 'Price2'),
         ('Price1', 'Price3'),
         ('Price2', 'Price3')]

for i, (p1, p2) in enumerate(pairs):
    rolling_corr = returns[p1].rolling(window).corr(returns[p2])
```

```
    plt.figure()
    plt.plot(rolling_corr)
    plt.title(f"Rolling Correlation: {p1} vs {p2}")
    plt.show()


#Now we create a GARCH model for each asset
models = {}
std_resid = {}

for col in returns.columns:
    model = arch_model(returns[col]*100, vol='GARCH', p=1, q=1, dist='t')
    res = model.fit(disp='off')

    models[col] = res
    std_resid[col] = res.std_resid

    #Plot conditional volatility
    plt.figure()
    plt.plot(res.conditional_volatility)
    plt.title(f"Conditional Volatility: {col}")
    plt.show()

#We convert residuals to DataFrame
std_resid = pd.DataFrame(std_resid)


#Check cross-asset dependence AFTER removing GARCH effects
print("\nCorrelation of standardized residuals:")
print(std_resid.corr())

print("\nCorrelation of squared standardized residuals (remaining volatility
link):")
print((std_resid**2).corr())


#Now we will do a Ljung-Box test to analyze the residuals
for col in std_resid.columns:
    print(f"\nLjung-Box test for {col}:")

    print(acorr_ljungbox(std_resid[col], lags=[10, 20], return_df=True))
    print(acorr_ljungbox(std_resid[col]**2, lags=[10, 20], return_df=True))


#Cross-correlation of squared returns (dynamic spillovers)
from statsmodels.tsa.stattools import ccf

for (p1, p2) in pairs:
    ccf_values = ccf(returns[p1]**2, returns[p2]**2)
```

```
    plt.figure()
    plt.plot(ccf_values[:20])
    plt.title(f"CCF Squared Returns: {p1} vs {p2}")
    plt.show()
```

Ex 3

```
#Exercise 3
#Importing the required packages for the analysis
from statsmodels.tsa.api import VAR
from statsmodels.tsa.stattools import adfuller

#Load the excel file and the 3rd sheet
df = pd.read_excel('Sguera_Vincenzo.xlsx', sheet_name='Ex.3')

#Convert date and drop the first row (because of value = 0)
df['obs'] = pd.to_datetime(df['obs'], format='%YM%m')
df = df.set_index('obs')
df = df.drop(df.index[0])

#Checking if the format is correct
print(df.head())


#Now we check the stationarity (thorugh ADF test)
def adf_test(series, name):
    result = adfuller(series.dropna())
    print(f"\nADF Test for {name}:")
    print(f"ADF statistic: {result[0]}")
    print(f"p-value: {result[1]}")

for col in df.columns:
    adf_test(df[col], col)

data = df.copy()


#Now we may apply the VAR model estimation
model = VAR(data)

#Selecting optimal lag length
lag_selection = model.select_order(maxlags=12)
print("\nLag selection:")
print(lag_selection.summary())

#Choose lag
lag = lag_selection.aic
print(f"\nChosen lag: {lag}")
```

```python
#Fitting VAR
results = model.fit(lag)

print("\nVAR Results Summary:")
print(results.summary())


#Now we do a stability check using the integrated function in python
print("\nIs VAR stable?")
print(results.is_stable())

#Cholesky identification (Sims approach)
irf = results.irf(20)

#Plotting IRFs
# Oil shock → all variables
irf.plot(orth=True)
plt.suptitle("Impulse Response Functions (Orthogonalized - Cholesky)", y=1.02)
plt.show()

# Focus specifically on oil shock propagation
print("\nIRF: Oil shock → Inflation")
irf.plot(orth=True, impulse='oil', response='inf')
plt.show()

print("\nIRF: Oil shock → GDP growth")
irf.plot(orth=True, impulse='oil', response='growth')
plt.show()


#Variance decomposition
fevd = results.fevd(10)

print("\nForecast Error Variance Decomposition:")
print(fevd.summary())
```